Custom System Call to Implement Publisher-Subscriber Inter-Process Communication at a Kernel Level

submitted in partial fulfilment of the requirements for the degree of

CS F372 : Operating Systems

by

Group 49

Name	***ID***	***Email***
Kevin Benny Mathew	*2023AAPS0143G*	*f20230143@goa.bits-pilani.ac.in*

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI
2026

os-systemcall-project

This project includes 4 system calls that are added to the linux kernel and recompiled. (Currently working in kernel version 5.19.8 but will code itself will not be affected when porting to newer kernel versions.

The system calls enable **creation, deletion, publishing and subscribing to a topic** at a kernel level. This is to replicate ROS-like publisher-subscriber architecture for inter process communication.

All files can also be found at Github

Note on LLM usage and other additional information

First Implementation is entirely human written (only non-human code is auto-completed code in VSCode). Even though the first implementation is functional, the second and final implementation is LLM-enhanced helping me get rid of race conditions I was unaware of as well as the `sys_delete_topic` being entirely LLM generated. All documentation including README and pdf report generated from README are entirely human written.

OS version information : Fedora 36 VM running kernel version 5.19.8

Table of Contents

- os-systemcall-project
  - Note on LLM usage and other additional information
  - Table of Contents
  - Method
  - File Index
  - First Implementation
    - Problems
  - Final Implementation
    - How This Fixes Prior Problems
  - Screenshots of Each Step

Method

This method adds four custom system calls directly into the Linux kernel source tree. Each syscall is defined using the `SYSCALL_DEFINE` macro and registered in the system call table as follows: - `sys_create_topic` - creates a topic and registers it into the global linked list. - `sys_publish` - copies a message from the user space to the kernel buffer and wakes all the waiting subsribers. - `sys_subscribe` - blocks the calling process untill woken up by the publishing of a message then copies it to the user space. - `sys_delete_topic` - removes a topic to free its resources (added in final implementation).

File Index

- `mysystemcall/pubsub.c.old` - First implementation of system calls
- `mysystemcall/pubsub.c` - Final implementation of system calls
- `mysystemcall/Makefile` - Makefile for custom system calls
- `Makefile` - Makefile for entire kernel build with modified lines
- `syscall_64.tbl` - System call table with added system calls for x86 architecture
- `test_programs/publisher_test.c` and `test_programs/subscriber_test.c` - basic system call test with publishing to a predefined topic
- `test_programs/sub_test2.c` and `test_programs/pub_test2.c` - test file with topic id as argument

First Implementation

The first implementation covers the first 3 system calls. The topics are stored as a struct (pubsub_topic) in a global linked list. This is protected by a spinlock. Each topic has an integer ID, a message buffer, the buffer length, a wait queue for any sibscribers as well as a spinlock of its own for accessing the buffer.

`create_topic` validates ID, checks for dupilicates under the list lock, and then finally makes a new topic struct (allocates memory and initialises) and inserts it into the list.

`publish` copies the incoming message from the user into a new kernel buffer via copy_from_user and then swaps it into the topic under the topics spinlock, frees the old buffer, then calls `wake_up_all` to wake up all the sleeping subscribers.

`subscribe` looks up the topic ID, adds the calling process to wait queue and checks if a message already exists. IF not, `schedule()` lets the CPU perform other tasks until a publish wakes it up. At that time it will check fot pending signals and will copy mesage to user space using `copy_to_user`.

Problems

- **TOCTOU race in `create_topic`**: The lock is released between the duplicate check and `list_add`, meaning two racing processes can insert duplicate topics.
- **Use-after-free in subscribe/publish**: `find_topic` returns a raw pointer then releases the lock; a concurrent delete can free the struct before it is used.
- **Lost/spurious wakeups in subscribe**: `prepare_to_wait` is called once outside any loop, so spurious wakeups and signal arrivals fall through the condition check with no way to re-sleep.
- **`copy_to_user` called under a spinlock**: This can page-fault and sleep, which is illegal inside a spinlock.
- **Single message buffer**: Only the last message is kept; a subscriber calling twice gets the same message and cannot catch up on anything it

missed.

Final Implementation

The final implementation adds an `atomic_t refcount` field to `pubsub_topic`. Two helpers replace the original one: `find_topic` (raw, only safe under the lock) and `find_topic_get`, which bumps the count atomically while `topic_list_lock` is still held. Every syscall calls `find_topic_get` and pairs it with a `topic_put` on every exit path. `topic_put` only calls `kfree` once the count hits zero.

`create_topic` now runs `kzalloc` before taking the list lock (since `GFP_KERNEL` can sleep), then holds the lock across both the duplicate check and the `list_add` without releasing it in between.

`delete_topic` removes the topic from the global list under `topic_list_lock`, sets a deleted flag under the topic's own lock, calls `wake_up_all` so sleeping subscribers see the flag and return `-ENOENT`, then drops the list's reference with `topic_put`.

`subscribe` wraps `prepare_to_wait`, the condition check, and `schedule()` in a `for(;;)` loop — the standard Linux wait-event pattern. `prepare_to_wait` re-arms `TASK_INTERRUPTIBLE` at the top of each iteration so no wakeup between the unlock and `schedule()` gets lost. The message is `memcpy`'d into a staging buffer `kbuf` under the lock (kernel-to-kernel, never sleeps), the lock drops, then `copy_to_user` runs against `kbuf`.

`msg_buf`/`msg_len` are replaced with `ring_data[RING_SIZE]`/`ring_len[RING_SIZE]` and a monotonic `pub_seq` counter. Message N sits at slot `(N-1) % RING_SIZE`. `subscribe` gets a `p_seq` cursor argument, starts at 0, blocks until `pub_seq > *p_seq`, copies the next slot, and writes the new sequence number back. Looping on `subscribe` gives ROS-style delivery — every message in order, and a slow subscriber skips to the oldest slot still in the ring rather than reading stale data.

How This Fixes Prior Problems

- **TOCTOU fixed**: Duplicate check and `list_add` now happen in one uninterrupted critical section.
- **Use-after-free fixed**: `find_topic_get` pins the struct via refcount; `topic_put` on every exit path means it only frees when no one holds a reference.
- **Lost wakeups fixed**: `prepare_to_wait` inside a `for(;;)` loop re-arms before every condition check.
- **Sleeping under spinlock fixed**: Message staged into `kbuf` via `memcpy` under the lock; `copy_to_user` runs after the lock is released.
- **Ring buffer replaces single buffer**: Subscribers use a `p_seq` cursor to receive every message in order without missing data.

Screenshots of Each Step

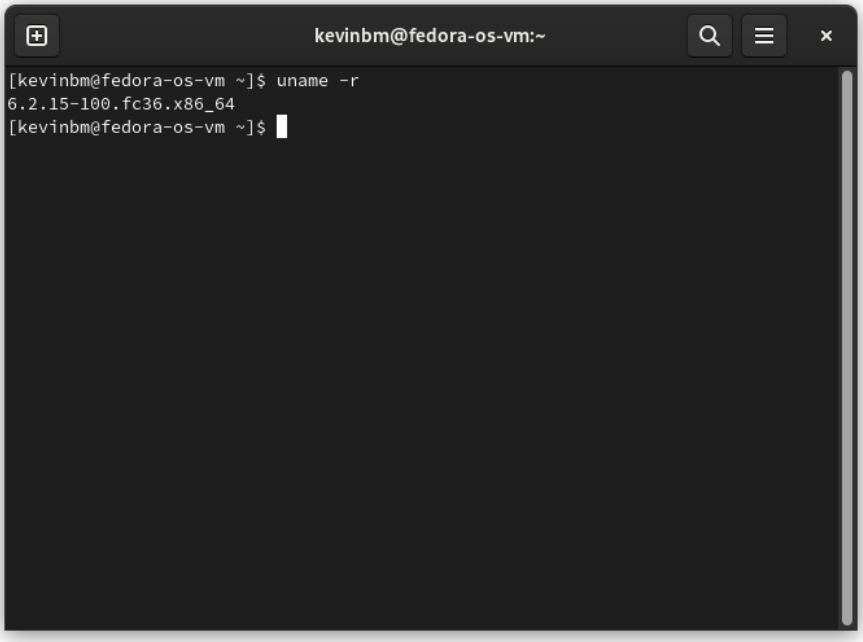

Figure 1: Kernel before Recompilation

```
[kevinbm@fedora-os-vm linux-5.19.8]$ make -j$(nproc)
  DESCEND objtool
  DESCEND bpf/resolve_btfids
  CALL    scripts/atomic/check-atomics.sh
  CALL    scripts/checksyscalls.sh
  CHK     include/generated/compile.h
  GENKEY  certs/signing_key.pem
/bin/sh: line 1: openssl: command not found
make[1]: *** [certs/Makefile:55: certs/signing_key.pem] Error 127
make[1]: *** Waiting for unfinished jobs....
make: *** [Makefile:1847: certs] Error 2
make: *** Waiting for unfinished jobs....
  CHK     kernel/kheaders_data.tar.xz
[kevinbm@fedora-os-vm linux-5.19.8]$ sudo make  modules_install
sed: can't read modules.order: No such file or directory
make: *** [Makefile:1478: __modinst_pre] Error 2
[kevinbm@fedora-os-vm linux-5.19.8]$
```

Figure 2: Building 5.19.8

```
  SIGN    /lib/modules/5.19.8/kernel/sound/usb/usx2y/snd-usb-usx2y.ko
  INSTALL /lib/modules/5.19.8/kernel/sound/virtio/virtio_snd.ko
  INSTALL /lib/modules/5.19.8/kernel/sound/x86/snd-hdmi-lpe-audio.ko
  INSTALL /lib/modules/5.19.8/kernel/virt/lib/irqbypass.ko
  INSTALL /lib/modules/5.19.8/kernel/sound/xen/snd_xen_front.ko
  SIGN    /lib/modules/5.19.8/kernel/virt/lib/irqbypass.ko
  SIGN    /lib/modules/5.19.8/kernel/sound/virtio/virtio_snd.ko
  SIGN    /lib/modules/5.19.8/kernel/sound/xen/snd_xen_front.ko
  SIGN    /lib/modules/5.19.8/kernel/sound/x86/snd-hdmi-lpe-audio.ko
  DEPMOD  /lib/modules/5.19.8

real    0m12.491s
user    1m11.204s
sys     0m33.218s
[kevinbm@fedora-os-vm linux-5.19.8]$ sudo make install
  INSTALL /boot
[kevinbm@fedora-os-vm linux-5.19.8]$
```

Figure 3: Installing modules and kernel

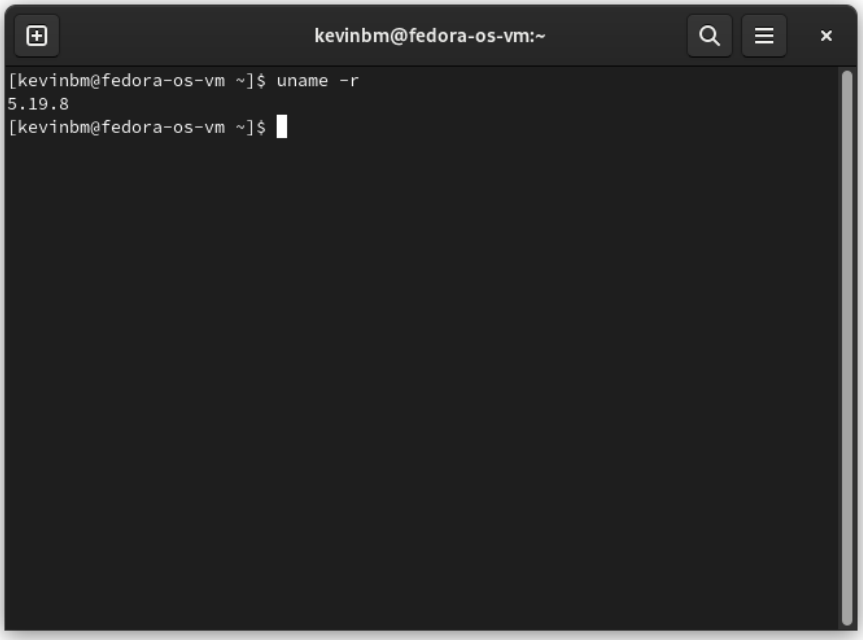

Figure 4: Sucessfully in 5.19.8

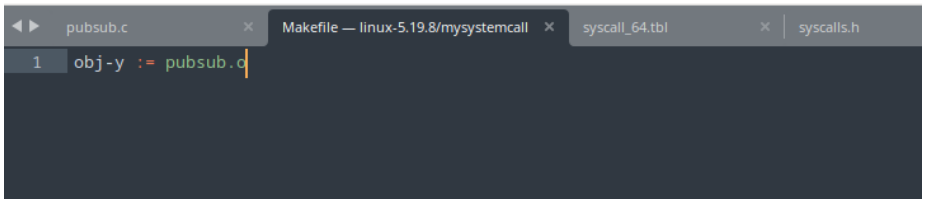

Figure 5: Makefile for pubsub.c

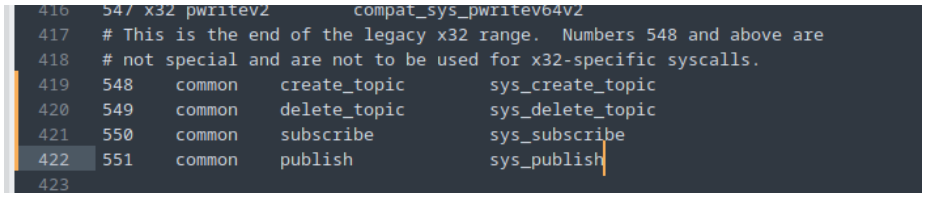

Figure 6: Modifying syscall table

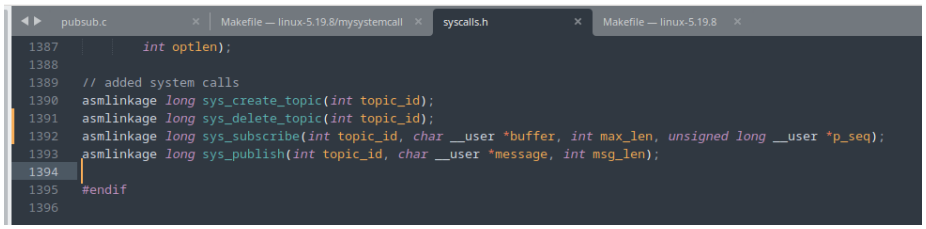

Figure 7: Adding prototype to the Syscall header

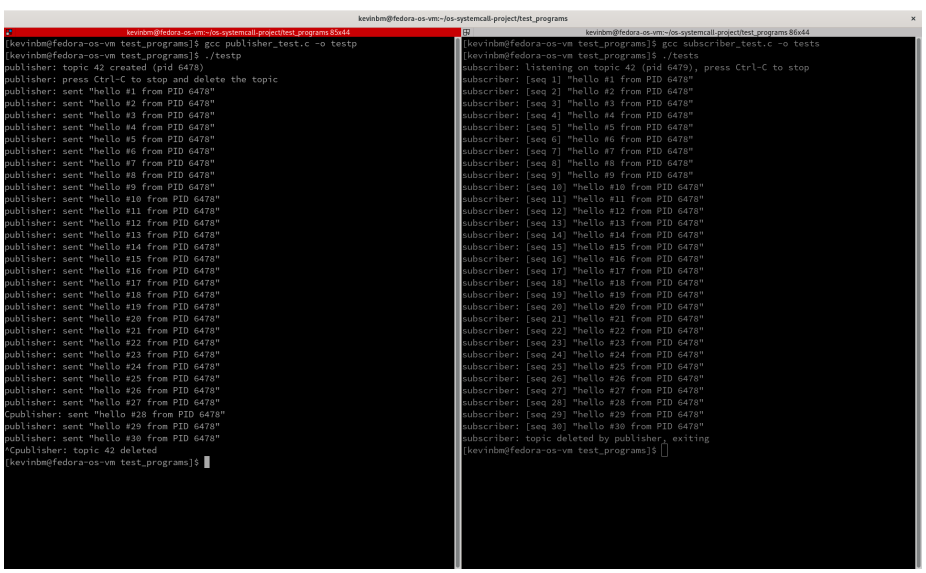

Figure 8: Basic test with pre-defined topic ID

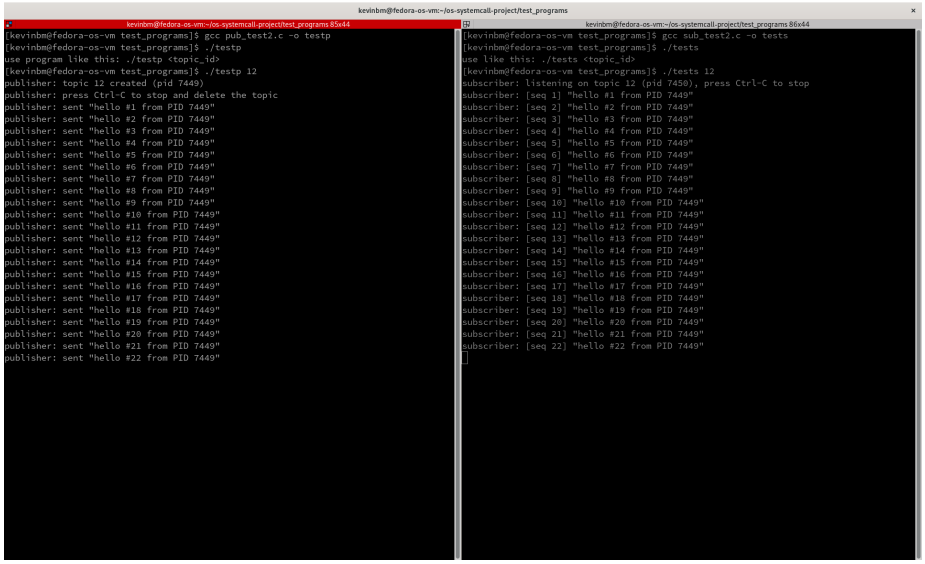

Figure 9: Test with user-input topic ID

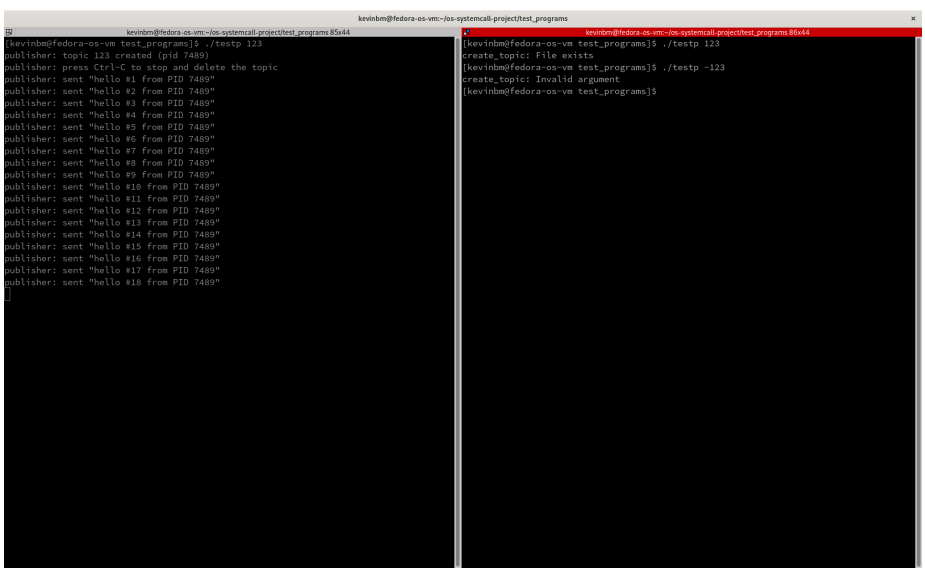

Figure 10: Showing Error management